A MINDMAP STUDIO GUIDE

Thinking in Maps

A practical guide to mind mapping — the technique and the canvas — with MindMap Studio.

Dann Bleeker Pedersen

Generated 2026-06-23
mindmap-studio.struktureretsundfornuft.dk

Contents

Foreword

Why a book about drawing bubbles and lines?

A mind map looks like the simplest thing in the world: a word in the middle, some branches, a few more words. People sketch them on napkins. So why a whole book?

Because the simplicity is the point, and the discipline behind it is easy to miss. A mind map is a thinking tool disguised as a doodle. It externalises the shape of an idea -- what depends on what, what sits beside what, what is still missing -- so your working memory stops juggling and your eyes can do the work instead. Done well, it turns a vague "I should plan this" into a structure you can argue with.

This book is in two voices at once. The first is about **the technique** : how to grow a map that actually helps you think, when to branch, when to stop, how to keep a big map from collapsing into noise. The second is about **the tool** : MindMap Studio , a small, fast, local-first app that runs in your browser, keeps your maps on your own machine, and asks for no account and no network. Every technique in the book is shown end-to-end in the app, with the exact keys and controls you press.

Who this is for

You don't need any prior experience with mind mapping, and you don't need to be a designer or a "visual thinker" -- a phrase that does more harm than good. If you can write a bulleted list, you can build a mind map, and this book will take you from a single node to a map you'd happily present.

If you're coming from MindManager, XMind, or a wall of sticky notes, you'll find the moves familiar and the friction lower. MindMap Studio is deliberately small: it does brainstorming and structuring well, and leaves project-management gravity to other tools.

How to read it

- **Part 1 -- Getting started** gets a real map on your screen and explains the few ideas the whole app rests on.
- **Part 2 -- Building maps** is the craft: structure, enrichment, and keeping large maps navigable.
- **Part 3 -- Sharing your thinking** covers getting the map *out* -- exporting, presenting, and running a session with other people.
- The **appendices** are reference: a keyboard card, the file-format details, and where to read more.

Read Part 1 in order. After that, jump to whatever you need. The map you build in Chapter 1 carries through the book, so it's worth following along in the app as you go.

Everything autosaves. You can't lose your place. Let's begin.

Your first map

From a blank canvas to a real structure in five minutes

Open MindMap Studio . The first time, you land on a sample map -- *Q3 Retail Plan* -- showing off notes, markers, a relationship arrow and a boundary. Click around it to get a feel for the canvas, then start your own: press **+ New...** in the toolbar and pick **Blank** .

You now have a single node that says *Untitled map* . That node is the **root** -- the one idea everything else hangs off. Double-click it (or just start typing) and give it a real subject. A map about planning a conference might have a root that simply says *Conference* .

The three keys that do everything

Almost all of map-building is three keys. Select the root, then:

- **Enter** adds a *sibling* -- another node at the same level. From the root, the first Enter gives you your first branch.
- **Tab** adds a *child* -- a node one level deeper, attached to the node you have selected. This is how a branch grows outward.
- **Delete** removes the selected node (and everything under it).

That's the whole loop: Tab to go deeper, Enter to stay at the same level, type to label, Delete to prune. Try it. Select *Conference* , press **Tab** , type `Venue` . Press **Enter** , type `Programme` . Press **Enter** , type `Budget` . You have three branches. Select `Venue` , press **Tab** , and add `Shortlist` , `Site visit` , `Contract` as its children. Within a minute you have a small tree.

> A fourth key is worth learning early: **Ctrl+Enter** inserts a line break *inside* a node, for when a label genuinely needs two lines. Use it sparingly -- short labels keep a map readable.

The keys are the fast path, but the canvas reaches back. Hover a node (or select it) and a small **+** appears on its right edge -- click it to add a child -- and a second **+** below it to add a sibling. Either way you land straight in the new node, ready to type, so a hand on the mouse never has to find the keyboard. The root only offers the child **+** : nothing sits beside the centre of a map.

There is also no separate "rename" step. To relabel any node, **double-click** it, press **F2** with it selected, or -- fastest -- just **start typing** : the first letter you press replaces the old label and the rest follows. The same three gestures work whether the node is brand new or one you made an hour ago.

Capturing in a hurry

The three keys are for *building* a map; sometimes you just need to *dump* something in before it escapes. Two shortcuts help. The **Quick add** box in the toolbar takes a topic and an **Enter** -- type, return, type, return -- and each line lands as a node without your touching the canvas: the fastest way to empty a head full of ideas into a map and sort them out later. And you can **drop** straight onto the canvas -- drag a link from your browser, or selected text from anywhere, onto the map and it

becomes a **floating topic** where you let go. Capture first, structure second; the dragging-into-shape of Chapter 3 is a calmer job once the ideas are already on the page.

Moving around

Click any node to select it; the arrow keys walk you between parent, child and siblings. Drag the canvas background to pan; scroll or pinch to zoom. If you ever lose the map off the edge of the screen, the **Fit** button in the toolbar frames the whole thing again.

One more canvas gesture earns its keep: **double-click an empty patch** of background and a fresh topic appears there, already in edit mode. It lands as a *floating topic* -- unattached to the tree -- which is exactly what you want when an idea arrives before you know where it belongs. Drop it down, name it, and connect it later; Chapter 3 covers the dragging-into-shape.

You can't lose it

MindMap Studio saves continuously to your browser's local storage (IndexedDB) as you type. Close the tab, reopen it tomorrow, and the map you were working on is exactly where you left it. There is no Save button because there is nothing to save -- it has already happened. There is also no server: the map never leaves your machine unless you explicitly export it, which is Chapter 6.

Undo is your safety net

Made a mess? **Ctrl+Z** undoes; **Ctrl+Y** (or **Ctrl+Shift+Z**) redoes. Undo is model-aware -- it reverses the actual change to the map, not just a visual tweak -- so you can experiment with structure freely and walk it back if a branch turns out wrong.

Home base: the Start screen

Once you've built a map or two, the **Start** button (top-left) takes you **home** -- a Start screen that's part launcher, part library. A **capture box** at the top turns a stray thought into a new map in one line; below it your saved maps sit as cards, alongside the **templates** and worked **examples** to start from, an **import** drop zone, and links to learn the app (this book included). It's the calm front door to the workbench -- somewhere to land, pick up an existing map, or start a fresh one, rather than always opening straight onto the canvas. Press **Start** any time to return; choose a map (or start one) and you're back on the canvas, where the rest of this book happens.

The very first time you open a fresh map, a small **"3 things to try"** card sits in the corner: double-click a topic to rename it, press **Tab** to add a child, press **Ctrl/Cmd + K** for anything. It is a one-time nudge -- close it with the **x** , or just make your first edit and it retires itself for good. You will not see it nagging on every map.

Now you try

Start a **Blank** map and build something real to you in five minutes -- a trip, a project, a decision you're weighing. Name the root, then grow three or four branches with a couple of children each,

using only **Tab** (deeper), **Enter** (same level), and typing. Delete a branch and **Ctrl+Z** it back, just to feel the safety net. Then close the tab and reopen it: the map is exactly where you left it. That loop -- Tab, Enter, type, undo, and it already saved itself -- is the whole foundation the rest of this book builds on.

Two captures worth trying once: from the **Start** screen, type a title into the capture box and press Enter -- you're on a fresh map in a single line; and while building, throw three half-formed ideas through the **Quick add** box (type, Enter, type, Enter) to feel how *capturing* and *structuring* are two different gears.

Try the mouse-only path too, so you know it is there when your hand is already on the trackpad. Hover a branch, click its **+** to add a child, and type the label that appears under your cursor. Then **double-click an empty patch** of canvas: a floating topic drops where you clicked, ready to name. Rename any node three ways to feel them -- **double-click** it, press **F2** , and on a third just **start typing** -- and notice you never had to reach for a menu.

That's enough to build maps. The next chapter slows down for a moment to explain *what* you've been building -- the handful of ideas the rest of the app rests on.

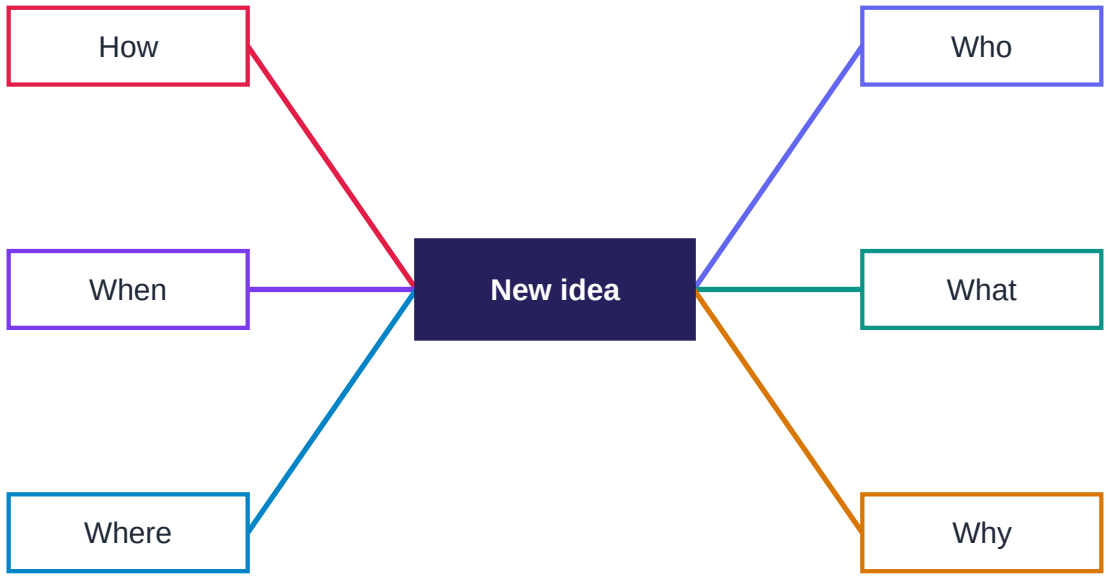
The anatomy of a map

A handful of ideas the whole app rests on

You can use MindMap Studio without ever reading this chapter. But ten minutes here will make every later feature feel obvious instead of arbitrary, because they all act on the same small set of parts.

The shape: one centre, many branches

A mind map is a tree with a single root in the middle and branches radiating outward. The classic starting point is a central idea surrounded by the six questions every plan eventually has to answer:



A mind map: one central idea, six branches — the Brainstorm template.

This is exactly what the built-in **Brainstorm** template gives you (you'll meet the other templates in Chapter 7). Each branch can grow its own children, and theirs can grow more, as deep as the thought goes. The radial shape isn't decoration: putting the subject in the centre keeps every branch equidistant from it, so no single line of thinking dominates just because it happens to be at the top of a list.

The node: the only real object

Everything on the canvas is a **node** . A node has:

- a **topic** -- the text you see;
- an optional set of **markers** -- small icons or tags (Chapter 4);
- an optional **note** -- longer prose attached behind the node (Chapter 4);
- an optional **image** ;

- and **children** -- the nodes hanging off it.

The root is just a node that happens to have no parent. A leaf is just a node with no children. There is no special "task" type or "milestone" type -- MindMap Studio keeps the model deliberately small. That smallness is why import, export and undo all behave predictably: there are fewer kinds of thing to get wrong.

Two things that aren't nodes

Two features draw on the canvas without being part of the tree:

- A **boundary** is a soft, shaded box drawn around a node and everything beneath it, grouping a branch visually ("everything in here is Phase 1").
- A **relationship** is an arrow drawn from one node to another, expressing a link that the tree structure can't -- a dependency, an influence, a "see also" across branches.

Both are covered in Chapter 3. The point for now: the tree carries the *hierarchy* , and these two carry the *exceptions* to it.

The canonical model

Under the surface, your map is a plain data structure -- a root node, its children, each node's topic and notes and markers. MindMap Studio treats that structure as the single source of truth. The colourful canvas is one *view* of it; the outline panel (Chapter 5) is another; an exported Markdown file is a third. Edit the map on the canvas and the underlying model updates; every other view follows.

This is worth internalising because it explains the app's most reassuring property: **what you see is always backed by real data you can take with you.** A map is never trapped in a proprietary blob. Chapter 6 shows you how to round-trip it through Markdown, JSON, OPML and more -- losslessly, in the case of JSON.

Now you try

Open the sample *Q3 Retail Plan* map and name the parts out loud: which node is the **root** , which are **branches** , where's the **boundary** , where's the **relationship** arrow. Then prove the "one model, many views" idea to yourself -- open the **Outline** panel (Chapter 5) and watch the same tree appear as an indented list, or export to **Markdown** (Chapter 6) and read your map as plain text. Same data, three faces. Now **rename a node on the canvas** and look again: the outline and the Markdown already show your edit, because the canvas writes straight to the one model every view reads from -- nothing to "save", nothing to sync.

With the parts named, the rest of the book is just learning what you can do to them.

Structuring ideas

Re-parenting, grouping, diagramming, and the links a tree can't hold

A first draft of a map is rarely in the right shape. A branch you started under *Venue* turns out to belong under *Budget* ; two ideas you thought were separate turn out to be the same. Restructuring is not a failure -- it *is* the thinking. This chapter is about moving things around without fear.

Drag to re-parent

To move a node -- and everything beneath it -- to a new home, just **drag it onto its new parent** . Drop `Catering` from under *Venue* onto *Budget* and the whole sub-branch travels with it, re-coloured to match its new branch. There is no cut-and-paste dance; the drag *is* the move. Because the underlying model updates atomically, **Ctrl+Z** puts it back if you misjudged the drop.

A good habit: build fast and loose first -- get every idea onto the canvas as a node, wherever -- then spend a second pass dragging things into the structure that emerges. Trying to get the hierarchy right while you're still generating ideas slows both down.

Copy a branch -- here, or in another map

Dragging *moves* a branch; sometimes you want a *copy* -- the same sub-tree in two places, or a structure you built in one map reused in another. **Right-click** a topic and choose **Copy branch** ; then right-click where it should go and choose **Paste branch here** to graft a fresh copy -- the node and everything beneath it -- under that parent. Every pasted node gets a new identity, so the copy is genuinely independent: edit it and the original doesn't budge.

The quietly useful part is that the clipboard **crosses maps** . Copy a branch in your *Q3 Plan* , open your *Q4 Plan* , and paste it in -- a standard checklist, a risk list, a team structure you keep re-using no longer has to be rebuilt each time. It's the middle ground between the drag (which only moves things within one map) and a full file import (which brings in a whole map): a way to carry just the piece you want from one map to the next.

Layout direction

By default MindMap Studio balances branches on both sides of the root. For some maps a one-sided layout reads better -- an agenda or a process that flows top-to-bottom in your head often wants everything going **right** . The layout control in the toolbar switches between **both** , **right** , and **left** . It's a view setting; it changes nothing about the structure, so flip it freely to see which framing helps.

Boundaries: grouping a branch

Sometimes a branch needs to be visibly *a thing* : "everything in here is out of scope", "this cluster is Phase 1". Select the node at the top of the branch and click **Group** : a shaded, rounded **boundary** box is drawn around it and all its descendants, and double-clicking the box's label chip lets you

name it. The boundary follows the branch as you edit it, so it keeps enclosing the right nodes even after you add or move children.

Boundaries are the right tool when the grouping is *hierarchical* -- it lines up with a single branch. When the grouping cuts across the tree, you want the next feature instead.

Relationships: the arrows across the tree

The tree is good at "X is part of Y". It is silent about "X depends on Y" when X and Y live on different branches. That's what **relationships** are for: draw an arrow from one node to another, anywhere on the canvas, and you've recorded a link the hierarchy couldn't express. A risk on the *Budget* branch that threatens a deliverable on the *Programme* branch; a decision that unblocks three others; a "see also". Use them sparingly -- a map laced with dozens of arrows is as hard to read as no structure at all -- but a handful of well-placed relationships often carry the most important information on the page.

> **Keep the tree honest.** Before drawing a relationship, ask whether the two nodes are really on different branches, or whether one should simply be re-parented under the other. An arrow is the right answer for genuine cross-links; it's the wrong answer for a hierarchy you just haven't tidied yet.

Once you have a few of these arrows, some of them will inevitably cross. Where two lines overlap, the eye can't tell whether they pass over each other or join -- and a false junction quietly tells the wrong story. Turn on **Line jumps** in the toolbar and the problem disappears: at every crossing, one of the two arrows lifts into a small **hop** over the other, the way a well-drawn wiring diagram or transit map keeps its lines legible. It's a per-map switch, so leave it off for sparse maps and flick it on the moment your arrows start to tangle; either way the hops are baked into your exports exactly as you see them on screen.

Floating topics

Not everything has to connect to the root. A **floating topic** is a node (or small sub-tree) that sits on the canvas unattached -- a parking lot for ideas you're not ready to place, a caption, a note-to-self. MindMap Studio renders floating topics imported from other tools in a labelled "Floating topics" branch.

A floating topic is a **first-class node** , not a second-class sticky: everything you do to a central topic, you can do here. Press **Tab** to add a child or **Enter** for a sibling; **Tab / Shift+Tab** indent and outdent (outdent a floating topic's child far enough and it becomes its own floating topic; indent one floating topic under another to nest them); right-click to **group it in a boundary** , **summarize** it, **paste a branch** beneath it, or **delete** it. Every inspector edit -- a note, a colour, task dates and progress, a hyperlink, markers and tags, a callout -- applies the same way. So a floating cluster is a genuine **staging area** : build it out as far as you like off to the side, then **drag it onto a branch** and the whole sub-tree joins the map. Drag a branch topic *out* and it floats free again.

Sticky notes

A floating topic is still a *topic* -- a node you might grow into a branch. Sometimes you want something lighter: a remark on the canvas that isn't part of the structure at all. The **Note** button drops a **sticky note** -- a free-floating amber card you can drag anywhere and type into. Use it for the things that hover *around* a map rather than *in* it: a "finish before Friday" to yourself, a caption over a cluster, a question for whoever you hand the map to. It's the canvas equivalent of a Post-it stuck to the whiteboard -- visible, movable, and plainly a note rather than a part of the tree. (For a remark pinned to *one specific node* , reach for a callout instead, Chapter 4; the sticky note is the free-floating kind.)

Summary topics

A boundary draws a box *around* a branch; a **summary** draws a labelled bracket *beside* one and says what it adds up to. Select the node at the top of a branch, click **Summary** , and a curly brace appears alongside it with an editable label -- "three options", "Q3 total", "needs sign-off". Where a boundary says *these belong together* , a summary says *here's the conclusion* . It's side-aware, so on a two-sided map the bracket sits on the outer edge where it reads naturally, and double-clicking its label renames it. Reach for a summary when a branch has a punchline -- a total, a verdict, a next step -- you want on the page without adding another node.

Editing an overlay: the inspector

Boundaries, summaries, and callouts aren't just drawn-and-forgotten -- each is a thing you can come back to. **Click** any of them once and the right-hand panel switches to an **overlay inspector** for that object. There you rename it (the box's label, the bracket's caption, the callout's text), re-tint it from a small set of swatches -- the colour carries through to every export -- and, for a boundary, switch its shape (rounded, square, ellipse, cloud, polygon) and its outline (solid, dashed, dotted). A **Delete** button at the foot removes the overlay without touching the nodes underneath; the branch and its topics stay exactly where they were.

So if a Phase 1 boundary you drew earlier should now read as out-of-scope, click it, dash its outline and recolour it grey -- the grouping is the same, the signal is softer. And when a summary bracket has served its purpose, click it and delete it; the conclusion goes, the branch remains.

A vocabulary of shapes

By default every topic is a soft rectangle, and for most maps that's exactly right -- the *words* carry the meaning. But when a map is really a **process or a decision flow** , shape becomes meaning. Select a node, open the **Style tab** (in the Info panel), and you can recast it as a **diamond** (a decision -- "approved?"), an **oval** (a start or end point), a **parallelogram** (an input or output), a **hexagon** (a preparation step), or a **cylinder** (a data store). These are the classic flowchart shapes, and they read instantly to anyone who's seen one: a diamond *asks* , an oval *bookends* , a cylinder *stores* .

When the five classics aren't quite the picture in your head, the Style bar keeps going. A **trapezoid** marks a manual operation; an **octagon** says *stop* or *limit* ; a **document** -- a page with a softly waving bottom edge -- stands for a report or a printed output; a **callout** is a rounded speech bubble

for an aside or an annotation; a **star** flags the one node that matters most; and a **cloud** is the universal shorthand for a loose idea or an external system you don't control. The concave ones (the star and the cloud especially) quietly pull their text inward so a long label never spills past the outline. Every one of these is drawn the same way on the canvas and in every export, so a flowchart you build here looks like a flowchart in the PNG, the PDF, and the Office decks. Use a shape when the geometry adds information; leave the soft rectangle when the word is enough.

Free-canvas mode

Auto-layout is a gift -- it keeps a growing tree tidy so you never nudge boxes around by hand. But some pictures aren't trees, and for those the tidy reflow gets in the way. Toggle **Free layout** and the auto-arranger steps aside: now you can **drag any node anywhere** and it stays put, its position saved with the map. This is the mode for a whiteboard-style diagram -- a system sketch, a seating plan, a freeform cluster of ideas -- where *where* a node sits is part of what it means. Switch Free layout off again and the tree snaps back to its automatic shape, every hand-placed position still remembered for when you turn it back on. It's the escape hatch from the grid, not a replacement for it: most maps want the auto-layout, and the handful that don't want it badly.

Diagram backdrops: onion, funnel, Venn

Sometimes the *frame* carries the idea. The **Diagram** builder drops a geometric backdrop behind your topics for you to place them into: an **onion** of concentric rings (core to periphery -- values, then strategy, then tactics), a **funnel** of stacked stages narrowing to a point (a pipeline, a filtering process; **/+** changes the stage count), or a **Venn** of two or three overlapping circles (what's shared versus what's distinct). The backdrop is pure geometry -- it pairs naturally with Free layout above, since you're positioning topics into regions by hand -- and, like shapes, it's drawn identically on screen and in every export, so the diagram you build is the diagram you hand out.

A different layout for one branch

Chapter 5 covers the **Layout** dropdown, which re-flows the *whole* map into an org-chart, a timeline, a fishbone, and more. Occasionally a single *branch* wants a different shape from the rest -- an org-chart of the team hanging off an otherwise radial strategy map. Right-click the branch and choose **Branch layout** to give just that subtree its own arrangement. The override is sized as one block in the main layout so it never collides with its siblings, and overrides nest -- a branch inside the branch can have its own again. It's a precision tool; most maps never need it, but when one section is a fundamentally different kind of thing, this is how you let it look like one.

Now you try

Open your conference map (or any map with a few branches). Pick a leaf that sits under the wrong parent and **drag it** where it belongs -- watch the whole sub-branch travel and re-colour. Then choose one branch that's genuinely "a thing" and give it a **boundary** . Finally, find two nodes on *different* branches that depend on each other and draw a **relationship** arrow between them. Step back: the tree now carries the hierarchy, the boundary carries the grouping, and the arrow carries

the one link the tree couldn't. If you can't find a real cross-branch link, that's a good sign -- don't invent one.

Two more moves worth a try. Flip the **layout direction** to right-only and back -- same structure, a different shape on the page; keep whichever frames the map better. Then drag a branch topic *out* into open space to make it a **floating topic** -- a parking lot for an idea you're not ready to place -- and drag it back onto a branch to re-attach it. Structure isn't a cage; it's something you reshape as the thinking firms up.

If your map is more diagram than tree, try the diagramming tools too. Cap a branch with a **Summary** that states its punchline. Turn a node into a **diamond** from the **Style bar** and watch the decision read at a glance. Or toggle **Free layout** and drag a few nodes into a shape you choose by hand -- then turn it off and watch the tree snap back, your positions remembered. None of these replace the tree; they're there for the maps the tree alone can't draw.

Three more moves for the maps that need them. Right-click a single branch and choose **Branch layout** -- give just that sub-tree an org-chart while the rest of the map stays radial; the layouts compose, so one busy branch can read its own way without reshaping anything else. When a map is really a framework, open the **Diagram builder** and drop topics into an onion, a funnel, or a Venn backdrop -- the frame carries the meaning so the labels don't have to. And reach past the diamond: the shape menu also holds a trapezoid, octagon, document, star, and cloud, each drawn identically on the canvas and in every export. One last doorway -- when two far-apart topics refer to each other but aren't a true dependency, give one a **jump link** to the other; clicking it flies the canvas to the target, the connection without the arrow. (You can also drop a web link or a block of text straight onto empty canvas and watch it land as a floating topic, ready to place.)

If you drew a boundary or a summary above, **click it once** now: the inspector opens on the right. Rename it, pick a colour, and -- for a boundary -- try the dashed outline. Then **delete** one and watch the branch underneath stay put; the grouping was never holding the nodes, only enclosing them. Now draw a second **relationship** arrow that crosses the first, then turn on **Line jumps** in the toolbar. Where the two lines met as an ambiguous junction, one now lifts in a small hop over the other and the crossing reads cleanly. Toggle it off and the hop vanishes -- it's a per-map switch, so save it for the moment your arrows start to tangle. Either way, export the map and the hops come through exactly as you see them.

Two quick reuse moves to finish: **right-click a branch -> Copy branch** , then paste it under another node -- or even into a *different* map -- and notice the copy is independent of the original. And drop a **sticky note** somewhere with a reminder to yourself: a remark that rides *with* the map without becoming part of its tree.

With structure under control, the next chapter makes individual nodes carry more.

Enriching nodes

Notes, markers, images, and making a map look like it means it

A bare tree of labels is often enough. But a node can carry far more than its topic, and used with restraint, that extra detail turns a sketch into a document you can hand to someone else.

Notes: the prose behind the bubble

A topic should stay short. The paragraph it deserves goes in a **note** . Select a node, open the **Notes** panel, and write. Notes accept **Markdown** -- headings, bold and italic, bullet lists, inline `code` , and links -- and the panel shows a live preview, so the formatting you type is the formatting you get.

Notes are where a map stops being a brainstorm and becomes a brief. The node says *Vendor decision* ; the note records the three options, the criteria, and why you chose the one you did. The canvas stays scannable; the detail is one click away on whichever node owns it.

Markers: status at a glance

A **marker** is a small icon or tag pinned to a node. Open the **Markers** palette and click to toggle one on the selected node: a priority flag, a tick, a question mark, a face. Markers are how a map carries *state* without words -- a row of green ticks and one red flag tells a reviewer where to look before they've read a single label.

Pick a small vocabulary and stick to it. "Red flag means blocked, tick means done, question mark means undecided" is a convention a whole team can read at a glance; fifteen different icons used once each is just clutter.

> Markers imported from a MindManager `.mmap` file are mapped to the closest emoji, so a map you bring in from elsewhere keeps its visual cues rather than arriving as bare text.

When you select several nodes at once, the Markers palette switches to **bulk** mode and works across the whole selection. A marker chip reads its state from the group: it shows **lit** when every selected topic already carries that marker, and **dashed** when only some of them do. Clicking follows the obvious rule -- a lit chip *removes* the marker from all the nodes; a dashed or empty chip *adds* it to every node that lacks it, in a single undo step. So to flag a dozen blocked topics, lasso them, click the flag once, and it lands on all twelve; click it again and it's gone from all twelve. Tags behave the same way in bulk: select the cluster, type or click a tag, and `risk` is on every node in the group at once -- which is exactly what a conditional rule (below) is waiting for.

Tasks: when a topic is also a job

A marker says *this is urgent* ; a **task** says *this is work, and here's where it stands* . Open the **Info** panel and a node can carry three task fields: a **progress** slider (0 to 100%), a **priority** (high, medium, low), and **start** and **due** dates. Together they turn a branch of a plan into something you

can actually track.

Progress **rolls up** : set the leaves and a parent shows the average of its children, so the top of a branch reports how far the whole thing has come without your doing the sum. Due dates work the same way -- a topic past its date is flagged **overdue** on the canvas, so a plan that's slipping says so out loud. None of this turns MindMap Studio into a project planner; it makes a *map* enough of one that you don't have to leave it to see what's done, what's next, and what's late. (Chapter 5's filter and board then let you ask the whole map "what's overdue?" at once.)

Images

A node can hold an **image** -- a screenshot, a logo, a photographed whiteboard, a chart. Attach one to the selected node and it renders inline on the canvas. An image is worth a paragraph of note when the thing you're describing is itself visual.

Sometimes you don't have a file to hand -- you just want a small visual cue: a **star** on the idea you're proudest of, a **warning** triangle on the risk, a **flag** on the decision still open. For that, the **Info** panel keeps a built-in grid of **stickers** : twenty clean little illustrations -- star, heart, check and cross badges, lightbulb, target, rocket, lock, clock, fire, and more -- in one quiet accent colour so they read as a set rather than a circus. Click one and it lands on the selected node. Behind the scenes a sticker simply *becomes* that node's image, which is the whole trick: it renders on the canvas and travels into every export exactly like a picture you supplied yourself, with nothing new to learn. A node holds one image at a time, so picking a new sticker (or a real photo) swaps out the last. Used sparingly -- one or two on the nodes that carry the most weight -- a sticker pulls the eye to what matters without a word.

Attachments: files that travel with the node

An image renders *on* the node; an **attachment** rides along *with* it. From the **Info** panel you can attach a file to the selected topic -- a spec, a contract, a spreadsheet -- and it's stored inside the map itself, so it travels in the JSON export and is there even offline, one click from downloading again. Reach for an attachment when the source document matters but doesn't belong on the canvas: the node says *Vendor contract* , the file *is* the contract, kept with the idea it backs up.

Styling: shape, fill, border, font

Beyond markers, individual nodes can be **styled** : change a node's shape, fill colour, border, or weight; bump a topic's font size or colour; and switch its **font family** from the **Font** picker -- the default sans-serif, a **serif** for a quieter, document-like topic, or **monospace** for code, IDs, and anything that should line up. Styling earns its keep when it *encodes* something -- the three "decision" nodes all share a fill, the headline branch is bolder than the rest -- and costs you readability when it's merely decorative. Style to create a pattern the reader can rely on, not to make the map "pop".

Styles you can reuse, formatting that follows the data

Hand-styling one node at a time is a scalpel; two features in the **Styles** panel (the toolbar button, not the per-node Style bar above) make styling *systematic* .

A **named style** saves a look so you can reuse it. Dress one node the way you want -- fill, border, font, weight -- save it under a name, and from then on a click applies that exact look to any selected topic, in this map or any other. "Our decision nodes look like *this* " stops being something you redo by hand; the styles you save are kept app-wide, so a visual language you settle on follows you across the whole library.

Conditional formatting goes further and styles nodes *by what they are* . Write a rule -- "anything tagged `risk` gets a red border", "anything 100% complete goes grey", "every node with a marker turns amber" -- and the map applies it everywhere, live, and keeps applying it as you edit. Where a named style is a look you *apply* , a conditional rule is a look that *follows the data* : tag a new node `risk` and it goes red without your touching the Style bar. On a big, changing map that's the difference between formatting you maintain and formatting that maintains itself.

Rich text: emphasis inside a topic

Styling paints the whole node; sometimes you want to stress just a word or two *inside* the label. While editing a topic, **Ctrl+B** , **Ctrl+I** , and **Ctrl+U** bold, italicise, or underline the selection -- the same muscle memory as any editor. Reach for it sparingly: one bold word that names the decision, an italicised term you're defining. The plain text is kept underneath, so your outline and every flat export stay clean even when the canvas is dressed up.

Callouts: a note that points

A **callout** is a small sticky note pinned beside a node -- a caveat, an open question, a "revisit this" -- without promoting it to a child topic. Right-click a node, choose **Add callout** , then double-click the bubble to write in it. Unlike a note, which lives *behind* the node one click away, a callout sits *on the canvas* , visible at a glance and drawn into your image exports -- the right tool for the one remark a reader must not miss. Like markers, their power is in scarcity: a map speckled with callouts has none.

A boundary, a summary bracket, and a callout each carry their own **colour** . Select one and the inspector offers a small row of swatches plus a **Default** button; the colour you pick re-tints the whole object -- its outline, fill, and label together -- on the canvas and in every export, and **Default** drops it back to the theme's accent. Use it the way you use everything else here: to *encode* . Tint the "out of scope" boundary grey and the "this release" one green and a reader sees the split before reading a word; leave the rest on the default accent so the two coloured ones carry the meaning.

Themes: the whole canvas at once

Where per-node styling is a scalpel, a **theme** is a coat of paint for the entire map. The theme gallery offers a few presets -- **Light** , **Dark** , **Ocean** , **Sunset** -- each a coordinated palette for the background, branches and text. Dark themes read well on a projector in a dim room; light themes print cleanly. Switching theme never touches your content, so try a few and keep whichever helps

you see the map.

The **Canvas** colour control sits one notch below a theme: it overrides just the background of *this one map* , leaving the theme's branch and text palette intact. It's a quiet but useful signal when you keep many maps -- a faint green wash on the "ideas" map, a warm one on "risks" -- so you know which map you're in at a glance. The colour saves with the map and follows it into an image or PDF export; the clears it back to the theme.

Right beside it, the button goes a step further and lets you drop a whole **image** behind the map -- a faint grid, a watermark, a photograph of the whiteboard you're rebuilding, your team's brand backdrop. Pick a file and it fills the canvas behind every topic, sitting on top of the background colour (so a transparent PNG lets that colour glow through the gaps). The picture is shrunk to a sane size and tucked inside the map itself, so the map stays a single portable file you can open offline -- and, like everything else here, what you see is what you get: the backdrop travels into your SVG, PNG, PDF and HTML exports unchanged. Use it sparingly, though; a busy photo behind your topics fights the words for attention, and the map is there for the words. The second lifts the image back off.

A note on restraint

Every feature in this chapter can be overused. The test is always the same: *does this make the map easier to think with?* A note that captures a decision -- yes. A marker convention a team shares -- yes. Six fonts and a gradient on every node -- no. The most useful maps are usually the plainest ones with enrichment applied exactly where it carries meaning.

One quiet piece of bookkeeping needs no decision from you at all. Select a node and the inspector header shows, in faint text under the breadcrumb, when the topic was **created** and last **modified** -- "created 3 d ago, modified 2 h ago", and a plain date once a change is more than a week old. You never set these; the map keeps them. They earn their place when a map outlives the meeting that made it: glancing at a branch and seeing it hasn't been touched in a month tells you whether it's settled or stale before you reopen the argument.

Now you try

Take a node that's carrying too much in its label and move the detail into a **note** (open the Notes panel and write a sentence or two in Markdown). Shorten the topic to a handful of words. Then agree a tiny **marker** convention with yourself -- say, a flag for "blocked" and a tick for "done" -- and apply it to three or four nodes. Read the map again: the canvas should be more scannable than before, with the depth one click away. If you reach for a fifth marker colour, stop -- the small vocabulary is the point.

Now add a layer of meaning. Attach an **image** to one node -- a screenshot or a logo -- and watch it render inline. Pick three related nodes (say, the ones that represent decisions) and give them a shared **fill** from the Style bar so the pattern reads at a glance; bump your headline branch's **font** up a size so the eye starts there. Finally, open the **theme** gallery and switch between Light and Dark -- your content doesn't change, only its coat of paint, so keep whichever helps you see the map. The rule throughout: styling that *encodes* something earns its place; styling that merely decorates

doesn't.

Finally, dress one topic up *inside* its label -- bold the single word that names what it is (**Ctrl+B** while editing) -- and pin a **callout** to the node you're least sure about, holding the question you still need to answer. Notice how differently the two read: the bold word is part of the idea; the callout hovers beside it, plainly a comment on the work rather than the work itself.

Finally, make a node carry *work* , not just words. In the **Info** panel give a topic a **due date** and drag its **progress** to half-done -- watch the parent branch show the rolled-up total and the canvas flag anything overdue. Then open the **Styles** panel, write one **conditional rule** (tag a node `risk` , have the rule paint it red), and add that tag to a second node: it turns red on its own. That's the chapter in miniature -- a node that tracks its own status, and formatting that follows the data instead of waiting for your hand.

Two more passes turn a handsome map into a working one. Give a topic a **priority** -- High, Medium, or Low -- then switch on the priority filter and watch everything but the High items fade; that is triage in two clicks. Pick a font that suits the map's register -- the Styles bar offers **Sans, Serif, or Mono** per topic -- and, once a node looks exactly right, **save its look as a named style** and drop that style onto its siblings so the set moves as one. Hang a real **file** off a topic -- a spec, a slide deck -- and it travels inside the map, downloadable whenever you reopen it. Reach into the **sticker** library when a small picture says more than a label would. And if the canvas itself wants a mood, give the whole map a **background colour** , or a faint background **image** behind everything -- it renders the same in every export, so what you arrange is what you ship.

One more pass, this time on a group. Select three or four nodes at once -- the Markers palette switches to **bulk** mode. Click a marker and watch it land on the whole selection; notice that a chip already on *all* of them reads **lit** (click to clear all) while one on only *some* reads **dashed** (click to add to all). Add a **tag** across the same selection the same way. Then pin a **callout** and open its inspector: give it a **colour** from the swatch row so the remark reads as deliberately set-apart, and try **Default** to drop it back. Finally, just look at a node's inspector header -- the faint **created** and **modified** line is there without your having lifted a finger, and on an old map that one glance tells you what's still alive and what has gone quiet.

The next chapter is about the opposite problem: when a map gets big, how do you keep finding your way around it?

Navigating large maps

Outline, find, collapse, and links between maps

A map with thirty nodes fits on a screen. A map with three hundred does not, and the features that felt optional at small scale become the difference between a map you use and a map you abandon. This chapter is about staying oriented.

Collapse and expand

Every branch can be **collapsed** to hide its children behind its parent, and expanded again to reveal them. Collapsing is how you control altitude: fold everything down to the top two levels to see the shape of the whole plan, then open just the branch you're working in. The toolbar's **collapse all** and **expand all** controls do it to the whole map at once -- collapse-all then open one branch is the fastest way to present a large map without overwhelming the room.

Fit

Lost? The **Fit** button reframes the entire map to the viewport in one click. It's the "take me home" of the canvas, and worth wiring into muscle memory: zoom in to work, hit Fit to see where you are.

The minimap and zoom

In the bottom-right corner sits a **minimap** -- a shrunk-down overview of the whole map, with a highlighted rectangle showing the slice you're currently looking at. On a big map it answers the question "where am I, and what else is out there?" at a glance. Click or drag inside it to jump the main view somewhere else: the rectangle follows your pointer and the canvas pans to match, so the minimap doubles as a fast way to travel across a map too large to scroll comfortably.

Below it are the **zoom controls** -- minus, a live percentage, plus, and a fit button. They do the same job as the mouse wheel but give you a precise readout and a deliberate step, which matters when you're lining a map up for a screenshot or a screen-share. The percentage tells you exactly how far in you are; the fit button is the same "take me home" as **Fit** above, kept within thumb's reach of the zoom buttons.

Layouts: the same map, different shape

The map's default shape -- a two-sided radial -- isn't the only way to read it. The **Layout** dropdown re-flows the *same* nodes into a different arrangement, and which one reads best depends on what the map *is* :

- An **org-chart** (top-down or bottom-up) suits a hierarchy: a team, a taxonomy, a decomposition.
- A **timeline** lays the first level out as a left-to-right sequence -- a roadmap, a process, the steps of an argument.
- A **fishbone** (the Ishikawa diagram) angles branches into a spine, the classic shape for cause-and-effect analysis.

- **Radial** fans every branch evenly around the root when you want the pure, balanced bloom.

Switching layout never changes your content -- only its geometry -- so it costs nothing to try a few and keep the one that makes the structure obvious. A backlog that felt tangled as a radial map can read as a clean timeline; a muddled list of problems snaps into focus as a fishbone.

The outline panel

The **Outline** panel shows your map as an indented list -- the same tree, read top to bottom instead of radiating outward. It's invaluable for three things: seeing the whole structure linearly, jumping straight to a node (click it in the outline and the canvas focuses it), and spotting structural problems a radial layout can hide, like a branch that's gone six levels deep while its siblings stayed shallow.

The outline has its own **filter** box: type a few letters and it narrows to matching topics, so a long map becomes a short list you can scan.

The marker and tag index

The outline answers "where is this *topic* ?" The **Index** panel (the **Index** button) answers a different question: "where is everything I *flagged* ?" It collects every marker and tag used anywhere in the map and lists them grouped by symbol -- a heading with the three topics you marked urgent under it, a heading with your favourites, and so on, each with a count. Click a topic in the index and the canvas jumps to it, exactly like the outline.

This is the read-only twin of the **Markers** palette. The palette is how you *put* a marker on the selected node; the index is how you *find* every node that already carries one. On a map with a hundred topics, that's the difference between scanning the whole thing for red flags and reading them off a list. Markers earn their keep precisely because the index makes them queryable after the fact -- so flag deliberately, and the index becomes a live status board.

Numbering the branches

A radial map is wonderful for thinking and terrible for *pointing* . "Look at the third sub-point of the second branch" is a sentence no one wants to say. The **1. Numbering** toggle fixes that: it stamps every topic with its outline number -- `1` , then `1.1` , `1.2` , then `1.2.1` -- on the canvas and down the outline at once, so now you can just say "see 2.3" and everyone's eyes land in the same place. The numbers follow into the PNG, PDF, and Office exports too, which is exactly when you need them: a printed map handed round a meeting, or a slide that references a node by number.

It's worth being clear about what numbering *isn't* . It doesn't change your map. The numbers are computed from the tree's shape the instant you switch it on and forgotten the instant you switch it off -- they never touch your topic text, so search still finds "Budget" and not "4.2 Budget", and a Markdown export is still clean prose. Reorder a branch and the numbers renumber themselves, because they were never really yours to begin with -- they belong to the structure. Turn it on when you need to talk *about* the map; turn it off when you just want to look *at* it.

Filtering without losing the forest

There's a moment with every large map where you stop wanting to *see everything* and start wanting to *see one thing* : every red flag, every node tagged `q3` , every topic that mentions "budget". The **Filter** panel does that without throwing anything away. Type some text, or click the marker and tag chips for what you're hunting, and the map answers by **fading everything that doesn't match** -- leaving the matches, and the branches that lead to them, at full strength. A count tells you how many topics qualified.

The phrase to hold onto is *read-only* . A filter here is a spotlight, not a pair of scissors: nothing is hidden, collapsed, or deleted, and the instant you hit **Clear** or close the panel the whole map comes back. That's deliberate. The fastest way to lose trust in a tool is to have it quietly remove something you needed; a filter that only changes opacity can never do that. The context stays too -- because the path from the centre to each match keeps its colour, you never get the disorienting "lone highlighted node floating in grey" that makes you forget where you are.

It pairs naturally with markers. Flag the risks as you build the map (Chapter 4), then, when the map is big and the meeting is short, filter to and read the risks straight off the lit branches. The markers are the *input* ; the filter is the *question you ask of them later* .

A filter you build often, you shouldn't have to rebuild. **Save** one as a named **preset** and it joins a list you re-apply in a click -- " risks", "due this week", "tagged `q3` " -- and because presets are kept app-wide, a filter that proves useful on one map is waiting on the next. The Power Filter stops being something you set up each time and becomes a set of saved questions you can ask of any map.

The filter answers "where is everything matching X?" Its close cousin, **Focus** , answers "show me just *this* ." Select a node, click Focus, and the map dims to that node's branch and the single path back to the centre -- the same spotlight-not-scissors idea, aimed at one branch instead of a query. It's how you walk a meeting through section 3 of a forty-node map without the other thirty-odd nodes competing for attention; **Esc** brings them back. Reach for Focus when you know *which* branch you mean, and the filter when you're asking the map a question across all of them.

Board view: the map as columns

A map is a tree; sometimes the question you have is a *board* question -- "what's in each bucket?" The **Board** button answers it without changing your map. It reads every topic's **tags** and lays the map out as **Kanban columns** , one per tag, each card showing the topic with its rolled-up completion and due date. Tag your nodes `todo` / `doing` / `done` (or `backlog` / `now` / `next`) and the very map you think in re-reads as a status wall you can scan.

It's **read-only** -- a lens, not a second copy. The board reads the tags; it doesn't move cards or rewrite the tree, and closing it leaves the map exactly as it was. Like the filter and the index, it's the same data asked a different question: the tree is how the work is *organised* , the board is how it's *progressing* .

Find and replace

Press **/** anywhere to jump straight to **Find** . It searches both topics *and* notes, so a term you only mentioned in a note is still findable. Matches are highlighted and you can step between them. Find is also **forgiving** : if what you typed matches nothing exactly, it quietly falls back to a typo-tolerant pass, so a half-remembered spelling -- *recieve* for *receive* -- still lands you on the node instead of an empty result. **Replace** turns find into a tidy-up tool: rename a project that got called three slightly different things, fix a term you decided to change, all in one pass.

> The `/` -to-find shortcut is the single most useful key on a big map. When you can't remember where something is, don't hunt -- press `/` and type.

The command palette: do anything by name

Find locates a *topic* ; the **command palette** locates an *action* . Press **Ctrl/Cmd + K** anywhere in the editor and a single search box opens over the map. Type a few letters of what you want -- `fit` , `collapse` , `board` , `outline` , `timeline` , `export pdf` -- and the list narrows to matching commands; arrow down to one and press **Enter** to run it. Every button on the toolbar is in here, so you never have to remember which row or menu hides a control: you just name it. The match is *fuzzy* -- it accepts a subsequence, so `cllpse` still finds **Collapse all branches** and `xpdf` still finds **Export .pdf** -- and the palette remembers your last few choices under a **Recent** heading, so the things you do often are one keystroke and Enter away the next time you open it.

The palette is selection-aware. Select a node first and a band of commands appears that act on *it* -- add a child, set a marker or priority, focus its branch, delete it -- so a node's whole context menu is reachable by name without touching the mouse. With nothing selected, those commands quietly drop off the list rather than offering you an action with no target.

Jump to any topic from the palette

The same **Ctrl/Cmd + K** box is also the fastest way to *travel* . Alongside the actions, every topic in the map sits in the list as **Go to: <topic>** ; type part of its text, press Enter, and the canvas selects and centres that node. The trick that makes it worth the keystroke is what it searches: each topic's row quietly folds in its **note text** as well, so a term you only wrote in a note -- never in a topic title -- still surfaces the right node. It is the `/` -to-Find idea widened to the whole map at once: one box that finds both the thing you want to *do* and the place you want to *be* .

The keyboard shortcuts cheat-sheet

When you forget a binding, you don't have to leave the map to look it up. Click the **? (help)** button in the icon rail -- or open the command palette and run **Keyboard shortcuts** -- and a cheat-sheet lists every editing, navigation, and view shortcut: Tab for a child, Enter for a sibling, `/` for Find, **Ctrl/Cmd + K** for the palette itself, and the pan-and-zoom gestures. The sheet is generated from the same shortcut table the app actually binds, so what it shows is always what the keys really do. Skim it once and the moves in this chapter stop being things you memorise and start being things you reach for.

Keyboard-reachable menus, and the same map on a phone

None of this assumes a mouse. The toolbar's dropdown menus and a node's right-click context menu are fully keyboard-driven: open one and the arrow keys walk the items, Home and End jump to the ends, Enter runs the highlighted one, and Escape closes without choosing. On a narrow screen the layout adapts rather than breaking: the side panels -- Outline, Index, Filter, Info -- slide up as a **bottom sheet** over a full-width canvas instead of crushing it into a sliver, so the same map you build at a desk stays usable on a phone.

Links: doorways between topics and maps

A radial map is a tree, but real subjects aren't: the risk you noted on one branch is the same risk that constrains a plan three branches away. Drawing a relationship line between them is one answer (Chapter 4); a **link** is the lighter one. Give a node a link and it grows a small -- click it and you travel.

A link can point three ways, all set from the **Info** panel. **Jump to a topic** points it at another *topic in the same map* , so "see also: Budget" becomes one click instead of a hunt -- the canvas leaps to that topic and selects it, no line cluttering the picture. **Link to a map** points it at *another map* in your library (Chapter 7): a node in your *Strategy* map can open your *Q3 Plan* map, which is how you build a small **atlas** instead of one unreadable continent -- a high-level map whose nodes are doorways into the detailed maps beneath them. And a plain web URL points it at a page, which opens in a new tab. A node holds one link at a time, and the follows whichever you set.

Search every map

Find searches the map you're in. Once your library grows into an atlas, **All maps** searches *all* of them at once -- every topic and note, floating topics included -- and picking a result opens that map and lands on the node. It's the companion to cross-map links: links are the doorways you placed on purpose, library search is for when you can't remember which map a thing is in.

Roll-ups: a node that mirrors another map

Cross-map links are *doorways* -- you click through to the other map. A **roll-up** brings the other map's content *to you* . Select a node, pick a source map from the **Roll-up** menu, and that node becomes a live mirror of the source: the source's top-level branches are copied in beneath it. Click **Roll-ups** whenever you like and every roll-up node re-pulls the latest from its source, so a high-level map can *aggregate* the maps below it instead of merely pointing at them.

This is the automated cousin of copy-paste (Chapter 3): a paste is a one-time snapshot you then own and edit; a roll-up stays **bound** to its source and refreshes on command. It's how you build a genuine **dashboard map** -- one overview whose branches are the current state of half a dozen detail maps -- and keep it current with a click rather than re-copying by hand. Treat a roll-up node as a window onto the source, not a place to edit: the mirrored branches are refreshed wholesale, so do the editing in the source map and roll it up again.

A working rhythm for big maps

Put together, the loop looks like this: **collapse all** to see the shape, open the branch you need, **/** to jump to a specific node, work, **Fit** to re-orient, repeat. The outline panel stays open on the side as a table of contents. None of these moves is clever on its own; the habit of using them together is what keeps a three-hundred-node map feeling as manageable as a thirty-node one.

Your workspace, remembered

The panels you work with -- the **Outline** on the side, the **Notes** editor -- stay how you left them. Close the app with the outline open and it's open when you come back; the app remembers your panel layout between sessions, so you don't rebuild your workspace every morning. It's a small thing, but it's the difference between a tool that settles into your habits and one you have to re-arrange every time you sit down.

Now you try

Find (or build) a map big enough that it doesn't fit on screen. **Collapse all** , then open just one branch and talk yourself through it. Press **/** and jump to a node you only mentioned in a *note* -- prove to yourself that Find searches notes, not just topics. Open the **Outline** panel and use its filter to narrow a long map to a short list. Zoom right in on one node, then press **Fit** -- watch the whole map snap back into view; that's your "take me home". If you keep more than one map, open **All maps** and search for a term you know lives in a *different* one -- watch it open that map and land on the node. While you have two maps, add a **cross-map link** : point a node in one at the other, then click it and watch the app hop maps -- that's how a high-level map becomes a set of doorways into the detailed ones beneath it. While you're here, flip the **Layout** dropdown between the radial default, an org-chart, and a timeline -- same nodes, three readings -- and keep whichever makes the structure clearest. Then **reload the page** : the Outline panel is exactly where you left it, because the app remembered your workspace. The goal isn't to memorise the controls; it's to feel how much calmer a big map gets when you drive it at the right altitude instead of staring at the whole thing at once. Two more lenses: tag a handful of nodes `todo` / `doing` / `done` and hit **Board** to read the same map as columns; and, if you keep an overview map, bind a node to another via the **Roll-up** menu and click **Roll-ups** to pull its branches in -- a dashboard that refreshes itself. Save any filter you liked as a preset while you're at it.

A few more ways to cut a big map down to what matters. Open the **marker and tag index** and click a tag -- it lists every node carrying it and jumps you there, no scrolling. Turn on **auto-numbering** and the branches read 1, 1.2, 1.3 like a document outline, so you can talk someone through the map by reference. When a single branch is all that matters for the moment, **focus** it and the rest dims to quiet context. The **Power Filter** does the same by rule rather than by hand -- dim, not hide, everything that lacks a marker, tag, or word, so the matches stand out while the map keeps its shape. Keep the **corner minimap** in view to hold your bearings while you are zoomed in, and don't fuss over a typo in **Find** -- a near-miss still lands the right node.

One more lens, the keyboard one. Press **Ctrl/Cmd + K** and first *do* something by name: type `collapse` and run **Collapse all branches** , then open the palette again and notice it now sits under **Recent** . Now *go* somewhere: type the text of a node you only mentioned in a *note* and watch **Go to:** land you on it, proving the palette searches notes too. Select a node and reopen the palette

-- see the band of actions that act on just that node, and watch them vanish when nothing is selected. Then click the **?** button (or run **Keyboard shortcuts** from the palette) and skim the cheat-sheet once. If you have a phone handy, open the same map there and toggle the **Outline** panel -- it rises as a bottom sheet over a full-width canvas instead of squeezing the map.

Part 3 turns outward: getting the map off your screen and in front of other people.

Sharing and exporting

Getting the map out -- losslessly when it matters

A map you can't get out of the app is a map held hostage. MindMap Studio is built on the opposite principle: your work is plain data, and there are many doors out. This chapter is the catalogue of them, and when to use which.

The export menu

The toolbar's **Export** menu offers, in rough order of fidelity:

- **JSON** -- the native format, and the only **lossless** one. Topics, notes, markers, images, boundaries, relationships, styling: everything in the model survives a round trip. Export to JSON for backup, for moving a map between machines, or any time you might want to re-open it later with nothing lost.
- **Markdown** -- the map as an indented outline (`#` root, nested bullets). Perfect for pasting into a document, a wiki, or a pull request. It's round-trippable as structure, though it naturally drops canvas-only detail like colours and arrows.
- **OPML** -- the interchange format outliners speak. Use it to hand your structure to another outlining tool.
- **FreeMind / Freeplane (`.mm`)** -- the most widely-read mind-map format. Carries the topic tree, links, folded branches, and notes, so a map opens in FreeMind, Freeplane, XMind, and the many tools that import `.mm` . The bridge to almost any other mind-mapper.
- **Mermaid (`.mmd`)** -- the `mindmap` text format you embed in Markdown, a README, or a wiki that renders Mermaid (GitHub, GitLab, Notion, many docs tools). For when the map should live *as text* inside something you're already writing.
- **XMind (`.xmind`)** -- the native format of one of the most popular mind-mappers, written the modern (2020+) way. Carries the topic tree, notes, links, and tags, plus floating topics and relationships, so the map opens natively in XMind rather than going through `.mm` .
- **SimpleMind (`.smmx`)** -- the native format of the cross-platform SimpleMind app. Carries the topic tree, notes, web links, and relations, plus floating topics, so the map opens natively in SimpleMind on desktop or mobile.
- **MindManager (`.mmap`)** -- MindManager's own format. Writes the topic tree, notes, hyperlinks, stock icons, relationships, and the two-sided left/right arrangement -- the mirror of the `.mmap` importer, so a map you started here can go back to a MindManager user.
- **PNG** and **SVG** -- the map as a picture. PNG for slides and chat; SVG when you want a crisp, scalable image that survives zooming.
- **HTML** -- a self-contained web page of the map, openable in any browser with nothing installed.
- **Interactive HTML** -- the same one-file, opens-anywhere idea, but *navigable* instead of a picture: the map becomes a collapsible, searchable outline. The recipient folds branches by clicking a topic, expands or collapses the whole thing at once, and types in a filter box to highlight and narrow to matching topics (plus Ctrl/-scroll to zoom and drag to pan). It's all one file with the data, styling, and a tiny script inlined -- no app, no server, no internet -- so it's the format for handing a *big* map to someone who needs to explore it, not just look at it.

- **HTML slide deck** -- the walk-through (Chapter 7) as a standalone, navigable presentation in a single file: an overview slide, then one per branch, advanced with the arrow keys or a click. Hand someone the file and they can present your map with nothing installed.
- **PowerPoint (`.pptx`)** -- a real, editable slide deck: an overview slide, then one per branch with its points as bullets. For when the deck has to live in PowerPoint -- a house template to apply, or a room to run from the corporate machine.
- **Word (`.docx`)** -- the map as an editable outline document: a title, indented bulleted topics, and notes as italic lines. For when the next step lives in a word processor -- minutes, a brief, a hand-off to someone who doesn't use the app.
- **Excel (`.xlsx`)** -- the map as an indented outline worksheet: each topic in the column matching its depth, with a Notes column. For when you want to sort, filter, or count the outline as a spreadsheet.
- **PDF** (via print) -- a fixed-layout document for sending and printing.

A rule of thumb: **JSON to keep it, Markdown to discuss it, PNG/SVG/HTML to show it, interactive HTML to let someone explore it, the slide deck or PowerPoint to present it, Word or PDF to send it, Excel to crunch it.**

A worked tour of the interchange exports, one file each. Take a project map and pick **Export -> OPML** ; open the `.opml` in your outliner and the topics arrive as nested headings, ready to keep editing as an outline. **Export -> FreeMind (`.mm`)** and double-click the file: it opens in FreeMind, Freeplane, or XMind with the tree, the folded branches, the links, and the notes intact -- the one bridge almost every mind-mapper reads. **Export -> Mermaid (`.mmd`)** gives you a `mindmap` block you paste straight into a README on GitHub, where it renders as a diagram in the rendered Markdown. **Export -> XMind (`.xmind`)** opens natively in XMind 2020+ -- tree, notes, links, tags, floating topics, and relationships -- with no `.mm` detour. **Export -> SimpleMind (`.smmx`)** opens natively in SimpleMind on desktop or phone, carrying notes, web links, and relations. And **Export -> HTML** hands someone a single self-contained page that opens in any browser with nothing installed -- one file, no app, no server. Same map, six more doors, and every one is a file on *your* disk.

Copy the outline

Sometimes you don't want a file at all -- you want the text on your clipboard, ready to paste. The **Copy outline** button copies the whole map as a Markdown outline in one click: drop it straight into an email, a chat message, a ticket, or a document. It's the same structure as the Markdown export without the round trip through a saved file -- the fastest way to turn a map into words somewhere else. Reach for it when the map was the *thinking* and some other place is where the writing has to land.

Importing

The same breadth applies going in. MindMap Studio reads:

- **Native** `.json` -- the lossless round trip of the export above.
- **Markdown** outlines -- any # /bullet structure becomes a map, **Markmap** -flavoured Markdown

(YAML frontmatter plus multi-level headings) included.
- **OPML** -- from other outliners.
- **FreeMind / Freeplane** `.mm` -- topics, links, folded state, and notes.
- **Mermaid** `mindmap` text -- any node shape; the hierarchy comes from the indentation.
- **XMind** `.xmind` (2020+) -- topics, notes, web links, and labels become tags.
- **SimpleMind** `.smmx` -- topic tree, notes, web links, and relations.
- **MindMup** `.mup` -- the JSON maps from the browser-based MindMup.
- **TextBundle / TextPack** (`.textpack`) -- the bundle's Markdown becomes the map; what Bear, Ulysses, and iA Writer export.
- **MindManager** `.mmap` -- see below.

You can **batch-import** several files at once, which turns a folder of outlines into a library of maps in one step. (The list keeps growing -- Word `.docx` , Excel `.xlsx` , iThoughts `.itmz` , MindMeister `.mind` , and older XMind files all open too.) These bridges cover the common ground between tools; each keeps the topic tree and the fields that map cleanly, and -- like the `.mmap` importer -- quietly leaves behind only the tool-specific extras it can't represent.

Not everything you want to map arrives as a file, though. Often it's just *text* -- an agenda in an email, a list in a chat, the bones of an outline you typed somewhere else. **Paste text** takes that straight from the clipboard: paste it in and the indentation (or # heading levels) becomes the tree, bullet and number markers are stripped, and you get topics. Drop it in as a new map, or **Add under selected** to graft it onto a branch you're already growing. It's the lowest- friction on-ramp there is -- and, because it never leaves the browser, the private way to bring in an outline you drafted anywhere, including one a chatbot wrote for you.

The MindManager bridge

If you're arriving from MindManager, the `.mmap` **importer** is the on-ramp. It reads a `.mmap` file and recovers the parts that map cleanly onto MindMap Studio's model: the topic tree and text, notes, stock icons (rendered as emoji markers), hyperlinks, relationships, boundaries, and floating topics. It was built against MindManager's own published schema rather than guessed at, so the common cases come through faithfully.

There is now also an **off-ramp** : **Export -> MindManager (** `.mmap` **)** writes the same parts back out -- tree, notes, hyperlinks, icons, relationships, and the two-sided left/right arrangement -- so a map you built here can land on a MindManager user's desk. It is the mirror of the importer, and a map round-trips back into MindMap Studio with those fields intact. MindManager is strict about its format, so confirm an exported file opens in your MindManager version before relying on it.

Both directions are deliberately **lossy on project data** : MindManager carries task scheduling, resources, and Gantt information that MindMap Studio doesn't model, and the importer tells you when it has left something behind rather than pretending the map came across whole.

> **Why lossy is the honest choice.** A converter that silently drops what it can't represent leaves you to discover the gaps later, usually at the worst moment. The importer's warnings are a feature: they tell you exactly what to check.

Going back in time

Exporting a `.json` is a snapshot you take on purpose; **version history** takes them for you. The **History** panel keeps a running list of past states of the current map -- captured quietly as you edit (every few minutes, not every keystroke) and whenever you click **Save version now** . Each entry shows when it was taken and how big the map was; **Restore** rolls the map back to it. The safety net under the safety net: restoring first checkpoints what you have *now* , so even an unwanted restore is one click from undone. It's all local (capped at the 30 most recent, kept in the browser's database) and travels with nothing -- a private undo that outlives the session, where the in-session Undo (Chapter 2) stops at the last reload. Think of Backup as the whole-library snapshot and History as the per-map flight recorder.

There's also a **Play timeline** button: instead of eyeballing the list, play the snapshots back in order and *watch* the map grow on the canvas -- step frame by frame, drag the scrubber to any point, or let it auto-play. It's read-only while you watch, so nothing changes until you pick a frame and hit **Restore this** (or **Exit** , or Esc, to drop back to the live map). On a map you've been building for weeks, it's a quietly satisfying way to see how the thinking took shape.

Where your data lives

Nothing in this chapter sends your map anywhere. Exports are files saved to *your* machine; imports read files *you* choose. Day to day, every map lives in your browser's local database, and the app works fully offline (Chapter 7). Sharing is always an explicit act, never a background one -- which is exactly how a thinking tool should treat the half-formed ideas you trust it with.

Because the app is a PWA, that locality holds even with no network. Load it once and it keeps working offline: the app shell is precached, so on your next visit -- plane, train, dead Wi-Fi -- it opens and your maps are right there, with a quiet **Ready to use offline.** note the first time the cache lands. When a new version ships, it never reloads under you mid-edit; instead a small **A new version is available.** toast appears with a **Refresh now** button, and the swap happens only when you click it. And on a phone the editor adapts -- the wide toolbar collapses into a single compact, horizontally-scrollable strip and the side panels slide up as bottom sheets -- so the same map is workable on a screen it was never drawn for.

Now you try

Take any map and export it twice: once to **JSON** and once to **Markdown** . Open the Markdown in a text editor — there's your outline, ready to paste into a document. Now start a **new** map and import the JSON you saved: it comes back exactly, down to the markers and arrows. You've just proved the thing that matters most about a thinking tool — your work isn't trapped in it. **JSON to keep it, Markdown to share it.** Make that round trip once and you'll trust the app with the ideas you're still figuring out.

Then take the same map to an audience two more ways. Export the **slide deck** (or **PowerPoint** , if the deck has to live there) and open it -- you're presenting: an overview, then one slide per branch, arrow keys to move, nothing installed. Export **Word** and open it in your word processor -- the same map as an editable outline, ready to become minutes or a brief. One map, four jobs -- archived,

discussed, presented, and written up -- and not once did your work leave your machine.

Now get it out as a *picture* and as a *paste* . Export **PNG** (or **SVG** for a crisp, scalable version) and drop it into a slide or a chat -- notice the labels, icons, arrows and boundaries all render, because the image carries real text, not a screenshot. Export to **PDF** when it needs to print, or to **Excel** when you want to sort and count the outline as a spreadsheet. For a big map someone needs to *explore* rather than glance at, export **Interactive HTML** and open it: same single offline file, but now you can fold branches and type in the filter box to narrow to what matters -- proof the export can carry the *navigation* , not just the picture. Finally, when you just need the words somewhere *now* , click **Copy outline** and paste -- no file, no download, just the map as a Markdown outline wherever your cursor is. Same map, every door out: a file to keep, a picture to show, a sheet to crunch, a paste to drop.

One safety net is worth feeling before you trust a map with real work: make a few edits, then open **version history** and restore an earlier snapshot -- the map rewinds intact, and you can **play the timeline back** to watch it grow from the first node to now. And when you bring a map in from MindManager, notice that its stock icons arrive as familiar **emoji** -- the meaning crosses over even when the file format doesn't.

Now prove the *interchange* . Export the map to **OPML** and reopen the `.opml` in your outliner -- the headings are your topics. Export **FreeMind (`.mm`)** and open it in Freeplane; export **Mermaid (`.mmd`)** and paste the `mindmap` block into a README so it renders as a diagram; export **XMind** or **SimpleMind** and watch it open natively in those apps, notes and links along for the ride; export self-contained **HTML** and double-click it in any browser. Then go the other way: take that `.mm` (or a `.opml` , `.xmind` , `.smmx` , `.mup` , `.itmz` , `.mind` , `.xlsx` , `.docx` , `.textpack` , a MindManager `.mmap` , a plain or Markmap-flavoured `.md` , or a Mermaid file) and **Import** it back -- the tree and the fields that map cleanly come across -- or drop a whole folder at once to **batch-import** them into a library. When you've only got loose text -- an agenda from an email -- **Paste text** turns the indentation into a tree in one click. Finally, feel the PWA: kill your network and reload (it still opens, maps and all, because the shell is precached), watch for the **A new version is available.** toast next time a build ships, and open the same map on your phone -- the toolbar shrinks to one scrollable strip and the panels become bottom sheets.

The final chapter is about the most demanding kind of sharing: standing up and walking a room through a map live.

Presenting and workshops

The library, templates, walk-through mode, and running a room

A map is often built to be shown. This chapter covers the features that turn a private canvas into a shared session: managing many maps, starting from the right template, presenting branch by branch, and the offline reliability that lets you do it anywhere.

The library: many maps, one app

MindMap Studio keeps a **library** of named maps in your browser. The map switcher in the toolbar moves between them; **+ New...** starts another; you can **duplicate** the current map (to try a variation without risking the original) and delete ones you're done with. Because each map is independent and locally stored, the library is your filing cabinet -- one map per project, per meeting, per idea -- with cross-map links (Chapter 5) tying related ones together.

Templates: a running start

A blank canvas is the right start for a freeform brainstorm. For everything else, **+ New...** offers templates:

- **Blank** -- a single root, for when the structure is entirely yours.
- **Brainstorm** -- the central idea with the six question-branches from Chapter 2.
- **SWOT** -- Strengths, Weaknesses, Opportunities, Threats, ready to fill in.
- **Project plan** -- Goals, Scope, Milestones, Risks, Team.

A template is just a starting structure you then make your own; its value is saving you the first thirty seconds of "what were the boxes again?" and nudging you toward a complete frame.

Where a template is an *empty* frame, the **Examples** group in the same **+ New…** menu gives you *complete* , worked maps -- a filled launch plan, a sprint retro, a worked SWOT, a trip itinerary, a study map -- to open and adapt rather than build from scratch. They're the fastest way to see what a finished map looks like, and to learn a feature by reading one that uses it.

Time-boxing a brainstorm

Ideas come faster under a gentle clock. The **brainstorm timer** in the toolbar is a small time-box: pick a length -- a few minutes -- start it, and generate hard until it rings. A divergent sprint ("every idea, no judging, go") works far better with a visible countdown than an open-ended "let's brainstorm", and a workshop gets a shared rhythm from it: sprint to capture, stop to cluster and prune, repeat. It's a nudge rather than a rule -- but the nudge is most of why time-boxing works.

Walk-through presentation mode

When it's time to present, **Present** enters a focused **walk-through** mode. Instead of showing the whole map at once and asking the room to find the thread, walk-through moves through the map

branch by branch, framing each in turn, so attention follows you. Combined with a dark theme (Chapter 4) and a collapsed starting view (Chapter 5), it turns a dense canvas into a guided tour: open the branch under discussion, talk to it, move on.

Presenting from the map itself -- rather than a deck exported from it -- keeps the single source of truth live. A question sends you to a different branch; an idea from the room becomes a node on the spot. The map is the slides *and* the working document.

Presenter view: what only you see

A slide is for the room. But you, the presenter, usually want more in front of you: the point you meant to make, where you are in the running order, and what's coming next. Press **P** (or click **Presenter view** in the control bar) and a sidebar opens beside the slide with exactly that -- visible to you, invisible to the audience, who still see only the slide.

Three things live in that sidebar. Your **speaker notes** come first: whatever you wrote in the current branch's note (Chapter 4) shows here, formatted, so your talking points travel with the map instead of on a separate sheet -- and if a slide has no note, it simply says so. Below that, **Next up** names the slide you're about to advance to, so you can set up the transition before you make it -- or see "End of map" and know to land the close. Last is the **Agenda** : the map of your whole talk, every slide in order with the current one lit and a "3 / 8" marker for your place in it. It's not just a readout -- click any line to jump straight there, which is how you handle the question that belongs three branches away and the "can you go back to that one?" without losing your footing.

It all sits on one screen -- there's no second window to wrangle -- and toggling it changes nothing for the room. Press **P** again to hide it. The habit worth forming: before you present, drop a sentence or two into the note on each branch you'll speak to. When the lights are on you, your script is already there.

It works offline, and it installs

MindMap Studio is a **progressive web app** . The first time you load it, it caches its own shell, so afterwards it opens and runs with **no network at all** -- on a plane, in a basement meeting room, anywhere. You can also **install** it to your desktop or home screen, where it launches in its own window like a native app. For a tool you might open to capture a thought the instant you have it, "always available, never loading" matters.

Because it caches itself, it also **updates itself politely** : when a new version ships, the running app shows a small "new version available -- Refresh now" prompt rather than reloading under you mid-thought. Click it when you're ready; an in-flight edit is never lost.

On the device in your hand

Because it's a web app, it goes where you do. On a **phone or tablet** the layout adapts -- the toolbar compacts to a scrollable strip and the side panels rise as a **bottom sheet** instead of squeezing the canvas -- so you can capture an idea or pull up a map on the move and open it on a laptop later. Same app, same maps, sized for the screen you're holding; paired with the offline caching above, it

makes "the map is wherever I am" simply true.

Back up the whole library

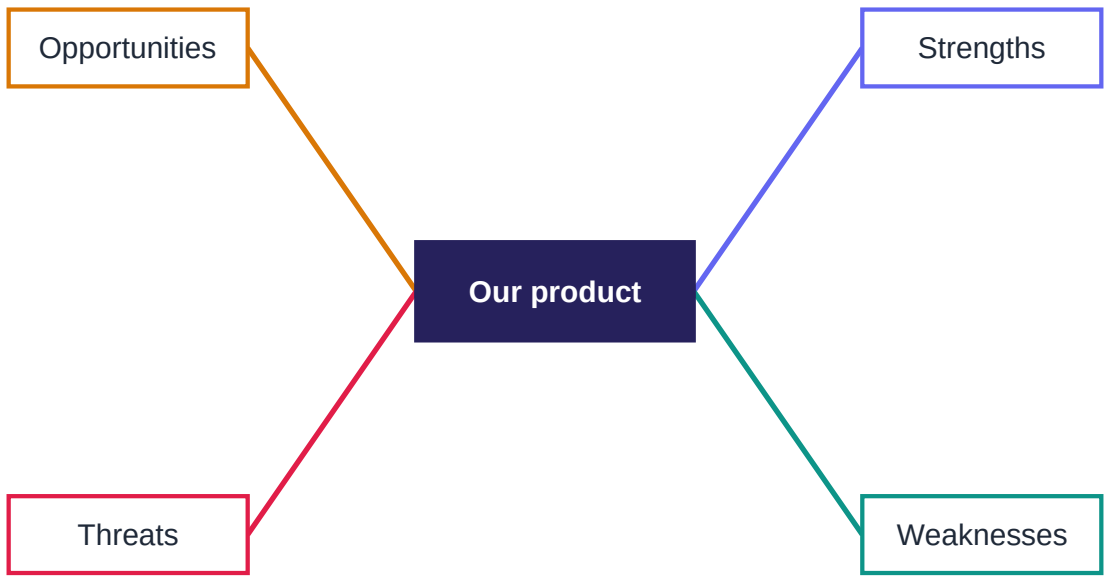
Individual maps export as JSON (Chapter 6). The whole **library** can be backed up and restored in one operation -- a single file with every map in it. Run a backup before a big reorganisation, before switching machines, or just on a schedule you trust. It's the belt to local storage's braces: your maps live on your machine, and a backup means a machine is not a single point of failure.

Running a session: a short playbook

1. **Before:** build or open the map; **collapse all** so it opens at altitude; pick a theme that suits the room.
2. **Open: Fit** , then enter **Present** .
3. **During:** walk branch by branch; capture new ideas as nodes as they come; use `/` to jump when the conversation does.
4. **After:** export to **PNG** or **PDF** for the recap; export **JSON** or run a **library backup** to keep the working map safe.

Now you try

Start a new map from the **SWOT** template. It opens with four branches ready to fill:



A SWOT map: a subject in the centre, four branches to fill in — the SWOT template.

Pick something real -- a product, a project, a decision -- put it in the centre, and spend ten minutes filling each branch. (Start the **brainstorm timer** for those ten minutes -- a visible countdown makes the sprint sharper than an open-ended "let's fill it in".) Don't aim for a long list; aim for the *honest* three items per branch. Then look across the four: does an Opportunity answer a Weakness? Does

a Threat undercut a Strength? Those cross-links are where a SWOT stops being four lists and starts being analysis -- draw a **relationship** arrow for each one you find. That move, more than the lists themselves, is the thinking.

Then rehearse the room. **Duplicate** the map so you can experiment freely, switch back to the original from the map switcher, and hit **Present** : walk-through frames each branch in turn -- arrow keys to move, the room's attention following yours, one branch at a time. Press **P** to flip on the **presenter view** -- your speaker notes, the next slide, and the agenda on your side of the screen only, the room still seeing just the slide. When you're done, run a **library backup** -- one file holding every map, the belt to local storage's braces -- and, if you haven't already, **install** the app to your desktop so it's one click away and runs with no network.

Prefer a running start over a blank SWOT? Open an **Example** instead (**+ New… -> Examples**) -- a filled launch plan, a retro, a trip, a worked SWOT -- and adapt it. Same skills, less blank page; it's also the quickest way to see a feature used in anger.

That's the whole tool, end to end -- from a single node in Chapter 1 to a map you can think with, enrich, navigate, share and present. The appendices that follow are the reference you'll come back to.

Appendix A -- Keyboard reference

The fastest way to build a map is to keep your hands on the keys. This is the full set, grouped by what you're doing.

Building structure

- **Enter** -- add a sibling (a node at the same level as the selected one).
- **Tab** -- add a child (a node one level deeper, under the selected one).
- **Ctrl+Enter** -- insert a line break *inside* the current node's label.
- **Delete** -- remove the selected node and everything beneath it.
- **Arrow keys** -- move the selection between parent, child and siblings.

Editing

- **Double-click** a node, or just start typing on a selected node, to edit its topic.
- **Ctrl+Z** -- undo the last change (model-aware: it reverses the real edit).
- **Ctrl+Y** or **Ctrl+Shift+Z** -- redo.

Finding and moving around

- **/** -- jump straight to Find (searches topics *and* notes).
- **Fit** (toolbar) -- reframe the whole map to the screen.
- **Drag a node** onto another to re-parent it (the whole sub-branch travels).
- **Drag the background** to pan; **scroll / pinch** to zoom.

Panels and views

- **Outline** (toolbar) -- toggle the indented-list view of the map.
- **Notes** (toolbar / node) -- open the Markdown note editor for the selected node.
- **Markers** -- open the marker palette to tag the selected node.
- **Collapse all / Expand all** (toolbar) -- fold or unfold every branch at once.
- **Layout** (toolbar) -- switch between both-sided, right-only and left-only.

> If you only memorise three things: **Tab** goes deeper, **Enter** stays level, and **/** finds anything. Everything else you can reach from the toolbar while you learn.

Appendix B -- Format reference

What goes in, what comes out, and what survives the trip. For the day-to-day guidance on *which* format to reach for, see Chapter 6; this appendix is the detail.

Export formats

- **JSON** (native) -- **lossless** . The complete model: topics, notes, markers, images, boundaries, relationships, styling. The format to use whenever you might re-open the map.
- **Markdown** -- the tree as an outline (`#` root, nested bullets). Round-trippable as structure; canvas-only detail (colour, arrows) is naturally dropped.
- **OPML** -- standard outliner interchange. Structure and topics; not canvas styling.
- **FreeMind / Freeplane (`.mm`)** -- the most widely-read mind-map format: topic tree, links, folded state, and notes. The bridge to most other mind-mappers.
- **Mermaid (`.mmd`)** -- the `mindmap` text format you embed in Markdown that renders Mermaid (GitHub, GitLab, Notion, many docs tools).
- **XMind (`.xmind`)** -- the modern (2020+) XMind package: topics, notes, links, tags, floating topics, and relationships.
- **SimpleMind (`.smmx`)** -- the native SimpleMind format: topic tree, notes, web links, relations, and floating topics.
- **PNG** -- a raster image of the canvas, for slides and chat.
- **SVG** -- a vector image: crisp at any zoom, ideal for high-resolution use.
- **HTML** -- a single self-contained page, openable in any browser.
- **HTML slide deck** -- the walk-through as a standalone, navigable presentation in one self-contained file (an overview slide, then one per branch).
- **PowerPoint (`.pptx`)** -- a real, editable slide deck (an overview slide, then one per branch with its subtree as bullets); a minimal PresentationML package that opens in PowerPoint, Keynote, LibreOffice, and Google Slides.
- **Excel (`.xlsx`)** -- the map as an indented outline worksheet (each topic in the column matching its depth, plus a Notes column); a minimal SpreadsheetML package that opens in Excel, LibreOffice Calc, Numbers, and Google Sheets.
- **Word (`.docx`)** -- the map as an editable outline document (title, indented bulleted topics, notes as italic lines); a minimal Open-XML package that opens in Word, LibreOffice, Pages, and Google Docs.
- **PDF** -- via the browser's print path; a fixed-layout document for sending or printing.

Import formats

- **JSON** (native) -- the lossless round trip of the JSON export.
- **Markdown** -- any `#` /bullet outline becomes a map, including **Markmap** -flavoured Markdown (YAML frontmatter plus multi-level headings).
- **OPML** -- outlines from other tools.
- **FreeMind / Freeplane** `.mm` -- topics, links, folded state, and notes.

- **Mermaid** `.mmd` -- `mindmap` text; the hierarchy comes from the indentation.
- **XMind** `.xmind` (2020+) -- topics, notes, web links, and labels (as tags).
- **SimpleMind** `.smmx` -- topic tree, notes, web links, and relations.
- **MindMup** `.mup` -- the JSON maps from the browser-based MindMup.
- **TextBundle / TextPack** (`.textpack`) -- the bundle's Markdown (`text.md`) becomes the map; what Bear, Ulysses, and iA Writer export.
- **iThoughts** `.itmz` , **MindMeister** `.mind` , **Word** `.docx` , **Excel** `.xlsx` -- topic tree (and notes, where the format carries them).
- **MindManager** `.mmap` -- one-way, lossy (see below).
- **Batch import** -- select several files at once to create several maps in one step.

What the `.mmap` importer recovers

The MindManager importer was built from MindManager's published XML schema, not guessed at. From a `.mmap` file it recovers:

- the **topic tree** and all topic text;
- **notes** (the plain-text preview MindManager stores);
- **stock icons** , mapped to the closest **emoji** marker;
- **hyperlinks** on nodes;
- **relationships** , as cross-links between nodes;
- **boundaries** drawn over a subtree;
- **floating topics** .

What it deliberately leaves behind

MindManager carries data MindMap Studio doesn't model -- chiefly **task and project information** : schedules, durations, resource assignments, Gantt data. The importer **warns** you about what it skipped rather than dropping it silently, and it does a left-behind check for any topic it couldn't reach. Treat the import as a migration -- a clean start in a new tool -- not as a two-way sync.

A practical note on lossless round trips

Only **JSON in and JSON out** is guaranteed to reproduce a map exactly. If a map matters, keep a JSON export (or a whole-library backup, Chapter 7) as the canonical copy, and treat the other formats as *views* of it -- excellent for sharing, not the thing you archive.

Appendix C -- Further reading

This book is deliberately practical -- enough technique to make the tool pay off, no more. If the *why* of visual thinking has caught your interest, here is where the deeper water is.

On mind mapping itself

The modern popular form of the mind map -- a central image, radiating branches, colour and keywords -- was popularised by **Tony Buzan** , who argued that this "radiant" layout mirrors how association actually works better than a linear list does. His books are the canonical starting point, and worth reading even where you end up disagreeing with the stricter rules he proposed. ("Mind Map" is a term associated with the Buzan Organisation; MindMap Studio uses the phrase descriptively for the diagram type and is an independent project.)

On externalising thought

The broader idea -- that thinking gets better when you get it *out of your head* and onto a surface you can rearrange -- runs through a lot of good writing on note-taking, sketching and "thinking on paper". Look into the literature on **visual note-taking** and **sketchnoting** for the hand-drawn tradition, and the **personal knowledge management** community for the digital one. Mind mapping sits between them: more structured than a sketch, more spatial than an outline.

On structure and decomposition

A mind map is one way to break a big thing into parts. If you find yourself wanting more rigour about *how* to decompose -- mutually exclusive branches, collectively exhaustive coverage, cause-and-effect rather than mere grouping -- the worlds of **issue trees** , **logic trees** and **causal mapping** are the next step out. They trade some of the mind map's freedom for more disciplined structure, and the two skills reinforce each other.

On the tool

MindMap Studio is open source. The application code is licensed under Apache 2.0; this book under Creative Commons BY-NC 4.0. The repository, the issue tracker, the live app, and a dashboard of the project's own metrics are all linked from mindmap-studio.struktureretsundfornuft.dk . If a chapter here describes something the app no longer does exactly that way, the repository is the source of truth -- and a good place to report the drift.

The shortest possible reading list

If you read only one more thing: pick a real problem you're avoiding, open a blank map, put the problem in the centre, and spend twenty minutes branching it. The technique teaches itself faster than any book can, this one included. The reading is for *after* you've felt it work.